

EXP.NO-4	EDA-Data Inspection and Analysis

AIM:

To inspect and analyze a dataset using Pandas by viewing DataFrames, filtering and examining data, and calculating measures of **central tendency** and **dispersion**.

PROCEDURE:

1. Import the Pandas library.
2. Load the CSV dataset using `pd.read_csv()`.
3. Display the first and last few records using `head()` and `tail()`.
4. Inspect the dataset structure using `info()` and `columns`.
5. Generate descriptive statistics using `describe()`.
6. Identify unique values in categorical columns.
7. Convert the Dependents column into a suitable numeric data type.
8. Display the frequency of values using `value_counts()`.
9. Calculate **mean, median, and mode** as measures of central tendency.
10. Calculate **range, variance, and standard deviation** as measures of dispersion.

PROGRAM:

```
import pandas as pd

df = pd.read_csv('/content/test_Y3wMUE5_7gLdaTN.csv')

print("First 5 rows:")

print(df.head())

print("\nLast 5 rows:")

print(df.tail())
```

```
print("\nDataset Information:")

df.info()

print("\nDescriptive Statistics:")

print(df.describe())

print("\nColumn Names:")

print(df.columns)

categorical_cols = ['Gender', 'Married', 'Dependents','Education',
'Self_Employed','Property_Area']

for col in categorical_cols:

    unique_vals = df[col].unique()

    print(f"\nUnique values in '{col}': {unique_vals}")

df['Dependents'] = df['Dependents'].replace('3+', 3)

df['Dependents'] = pd.to_numeric(df['Dependents'],errors='coerce').astype('Int64')

print("\nValue counts in 'Dependents':")

print(df['Dependents'].value_counts(dropna=False))

cols = ['ApplicantIncome', 'CoapplicantIncome','LoanAmount']

for col in cols:

    print(f"\nStatistics for '{col}':")

    print(f"Mean: {df[col].mean():.2f}")

    print(f"Median: {df[col].median():.2f}")

    print(f"Mode: {df[col].mode().values}")

    print(f"Range: {df[col].max() - df[col].min():.2f}")

    print(f"Variance: {df[col].var():.2f}")

    print(f"Standard Deviation: {df[col].std():.2f}")
```

OUTPUT:

```
First 5 Rows:
  Loan_ID Gender Married Dependents Education Self_Employed \
0 LP001002 Male No 0 Graduate No
1 LP001003 Male Yes 1 Graduate No
2 LP001005 Male Yes 0 Graduate Yes
3 LP001006 Male Yes 0 Not Graduate No
4 LP001008 Male No 0 Graduate No

  ApplicantIncome CoapplicantIncome LoanAmount Loan_Amount_Term \
0 5849 0.0 NaN 360.0
1 4583 1508.0 128.0 360.0
2 3000 0.0 66.0 360.0
3 2583 2358.0 120.0 360.0
4 6000 0.0 141.0 360.0

  Credit_History Property_Area Loan_Status
0 1.0 Urban Y
1 1.0 Rural N
2 1.0 Urban Y
3 1.0 Urban Y
4 1.0 Urban Y
```

```
Last 5 Rows:
  Loan_ID Gender Married Dependents Education Self_Employed \
609 LP002978 Female No 0 Graduate No
610 LP002979 Male Yes 3+ Graduate No
611 LP002983 Male Yes 1 Graduate No
612 LP002984 Male Yes 2 Graduate No
613 LP002990 Female No 0 Graduate Yes

  ApplicantIncome CoapplicantIncome LoanAmount Loan_Amount_Term \
609 2900 0.0 71.0 360.0
610 4106 0.0 40.0 180.0
611 8072 240.0 253.0 360.0
612 7583 0.0 187.0 360.0
613 4583 0.0 133.0 360.0

  Credit_History Property_Area Loan_Status
609 1.0 Rural Y
610 1.0 Rural Y
611 1.0 Urban Y
612 1.0 Urban Y
613 0.0 Semiurban N
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
# Column Non-Null Count Dtype
---
0 Loan_ID 614 non-null object
1 Gender 601 non-null object
2 Married 611 non-null object
3 Dependents 599 non-null object
4 Education 614 non-null object
5 Self_Employed 582 non-null object
6 ApplicantIncome 614 non-null int64
7 CoapplicantIncome 614 non-null float64
8 LoanAmount 592 non-null float64
9 Loan_Amount_Term 600 non-null float64
10 Credit_History 564 non-null float64
11 Property_Area 614 non-null object
12 Loan_Status 614 non-null object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
None
```

```
Descriptive Statistics:
  ApplicantIncome CoapplicantIncome LoanAmount Loan_Amount_Term
count 614.000000 614.000000 592.000000 600.000000
mean 5403.459283 1621.245798 146.412162 342.000000
std 6109.041673 2926.248369 85.587325 65.12041
min 150.000000 0.000000 9.000000 12.000000
25% 2877.500000 0.000000 100.000000 360.000000
50% 3812.500000 1188.500000 128.000000 360.000000
75% 5795.000000 2297.250000 168.000000 360.000000
max 81000.000000 41667.000000 700.000000 480.000000

  Credit_History
count 564.000000
mean 0.842199
std 0.364878
min 0.000000
25% 1.000000
50% 1.000000
75% 1.000000
max 1.000000
```

```
Column Names:
Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
      'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
      'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status'],
      dtype='object')
```

```
Value Counts in 'Dependents':
Dependents
0      345
1      102
2      101
3       51
<NA>    15
Name: count, dtype: Int64
```

```
Unique values in 'Gender':  
['Male' 'Female' nan]  
  
Unique values in 'Married':  
['No' 'Yes' nan]  
  
Unique values in 'Dependents':  
['0' '1' '2' '3+' nan]  
  
Unique values in 'Education':  
['Graduate' 'Not Graduate']  
  
Unique values in 'Self_Employed':  
['No' 'Yes' nan]  
  
Unique values in 'Property_Area':  
['Urban' 'Rural' 'Semiurban']
```

```
Statistics for 'ApplicantIncome':  
Mean: 5403.46  
Median: 3812.50  
Mode: [2500]  
Range: 80850.00  
Variance: 37320390.17  
Standard Deviation: 6109.04  
  
Statistics for 'CoapplicantIncome':  
Mean: 1621.25  
Median: 1188.50  
Mode: [0.]  
Range: 41667.00  
Variance: 8562929.52  
Standard Deviation: 2926.25  
  
Statistics for 'LoanAmount':  
Mean: 146.41  
Median: 128.00  
Mode: [120.]  
Range: 691.00  
Variance: 7325.19  
Standard Deviation: 85.59
```

RESULT:

Thus, the given program was successfully executed to inspect and analyze the dataset using Pandas and calculate measures of central tendency and dispersion.